

```

import subprocess
import sys

packages = ['pandas', 'numpy', 'matplotlib', 'seaborn', 'scikit-learn', 'scipy', 'statsmodels']
for package in packages:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", package])

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import pearsonr, spearmanr, shapiro, zscore, ttest_ind
from sklearn.preprocessing import LabelEncoder
import warnings
warnings.filterwarnings('ignore')

sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (14, 7)

print("="*90)
print("E-COMMERCE CUSTOMER BEHAVIOR - WEEK 5: STATISTICAL ANALYSIS")
print("="*90)

from google.colab import files
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

print(f"\nDataset: {filename}")
print(f"Dimensions: {df.shape[0]} customers x {df.shape[1]} features")
print("\nFirst 5 records:")
print(df.head())
print("\nData Info:")
print(df.info())
print("\nBasic Statistics:")
print(df.describe())

df_processed = df.copy()
for col in df_processed.columns:
    if df_processed[col].isnull().sum() > 0:
        if df_processed[col].dtype in ['int64', 'float64']:
            df_processed[col].fillna(df_processed[col].median(), inplace=True)
        else:
            df_processed[col].fillna(df_processed[col].mode()[0], inplace=True)

for col in ['Gender', 'Membership Type', 'Satisfaction Level']:
    le = LabelEncoder()
    df_processed[col + '_encoded'] = le.fit_transform(df_processed[col])

df_processed['Discount Applied'] = df_processed['Discount Applied'].astype(int)

print("\n" + "="*90)
print("SECTION 1: CORRELATION ANALYSIS (PEARSON & SPEARMAN)")
print("="*90)

```

```

correlation_pairs = [
    ('Age', 'Total Spend'),
    ('Items Purchased', 'Total Spend'),
    ('Average Rating', 'Total Spend'),
    ('Days Since Last Purchase', 'Total Spend'),
    ('Average Rating', 'Items Purchased')
]

print("\nPEARSON CORRELATION (measures linear relationships):")
print("-" * 90)

pearson_data = []
for var1, var2 in correlation_pairs:
    r, p_value = pearsonr(df[var1], df[var2])

    if abs(r) > 0.8:
        strength = "Very Strong"
    elif abs(r) > 0.6:
        strength = "Strong"
    elif abs(r) > 0.4:
        strength = "Moderate"
    else:
        strength = "Weak"

    direction = "Positive" if r > 0 else "Negative"
    sig = "Significant***" if p_value < 0.001 else "Significant**" if p_value < 0.01 else "Significant*" if p_value < 0.05 else "Not Significant"

    pearson_data.append({
        'Variables': f'{var1} ↔ {var2}',
        'Correlation (r)': f'{r:.4f}',
        'P-value': f'{p_value:.6f}',
        'Strength': strength,
        'Direction': direction,
        'Result': sig
    })

    print(f"{var1:25} vs {var2:25}")
    print(f"   r = {r:7.4f} | p = {p_value:.6f} | {strength} {direction} | {sig}")
    print()

pearson_df = pd.DataFrame(pearson_data)

print("\nSPERMAN CORRELATION (non-parametric, robust to outliers):")
print("-" * 90)

spearman_data = []
for var1, var2 in correlation_pairs:
    rho, p_value = spearmanr(df[var1], df[var2])

    if abs(rho) > 0.8:
        strength = "Very Strong"
    elif abs(rho) > 0.6:
        strength = "Strong"
    elif abs(rho) > 0.4:
        strength = "Moderate"
    else:
        strength = "Weak"

    direction = "Positive" if rho > 0 else "Negative"

```

```

sig = "Significant****" if p_value < 0.001 else "Significant***" if p_value < 0.01 else "Significant*" if p_value < 0.05 else "Not Significant"

spearman_data.append({
    'Variables': f'{var1} ↔ {var2}',
    'Correlation (ρ)': f'{rho:.4f}',
    'P-value': f'{p_value:.6f}',
    'Strength': strength,
    'Direction': direction,
    'Result': sig
})

print(f"{var1:25} vs {var2:25}")
print(f"  ρ = {rho:7.4f} | p = {p_value:.6f} | {strength} {direction} | {sig}")
print()

spearman_df = pd.DataFrame(spearman_data)

numeric_cols = ['Age', 'Total Spend', 'Items Purchased', 'Average Rating', 'Days Since Last Purchase']
fig, ax = plt.subplots(figsize=(12, 9))
numeric_df = df[numeric_cols]
corr_matrix = numeric_df.corr()
sns.heatmap(corr_matrix, annot=True, fmt='.3f', cmap='coolwarm', center=0,
            square=True, ax=ax, cbar_kws={'label': 'Correlation Coefficient'})
ax.set_title('Pearson Correlation Matrix - Numerical Features', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

print("\nINTERPRETATION OF CORRELATION RESULTS:")
print("-" * 90)
print("""
1. PEARSON vs SPEARMAN COMPARISON:
   - Similar values = Linear relationships are appropriate
   - Different values = Non-linear or monotonic relationships

2. STRONG POSITIVE CORRELATIONS (r > 0.6):
   - Items Purchased ↔ Total Spend: Strong positive (r ≈ 0.97)
     → As customers buy more items, they spend more
     → Items per transaction drives spending (key for models)

   - Average Rating ↔ Total Spend: Strong positive (r ≈ 0.94)
     → Higher satisfaction is associated with higher spending
     → Quality perception drives purchasing decisions

3. STRONG NEGATIVE CORRELATIONS (r < -0.6):
   - Age ↔ Total Spend: Strong negative (r ≈ -0.68)
     → Younger customers spend significantly more
     → Important demographic insight for targeting

4. STATISTICAL SIGNIFICANCE (p < 0.001):
   - All major correlations are highly significant
   - Not due to chance; represent true relationships
   - Can be used confidently in predictive models

5. PRACTICAL IMPLICATIONS:
   - Use Items Purchased as PRIMARY predictor for spending
   - Include Average Rating as SECONDARY predictor
   - Age must be included as demographic factor
   - Relationships are strong enough for linear models
""")

```

```

"""
print("\n" + "="*90)
print("SECTION 2: DIAGNOSTIC MEASURES (Distribution Analysis)")
print("="*90)

print("\nSKEWNESS AND KURTOSIS ANALYSIS:")
print("-" * 90)

diagnostic_data = []
for col in numeric_cols:
    skewness = stats.skew(df[col])
    kurtosis = stats.kurtosis(df[col])
    mean = df[col].mean()
    median = df[col].median()

    if abs(skewness) < 0.5:
        skew_type = "Approximately Symmetric"
    elif skewness > 0:
        skew_type = "Right-skewed (positive)"
    else:
        skew_type = "Left-skewed (negative)"

    diagnostic_data.append({
        'Variable': col,
        'Mean': f'{mean:.2f}',
        'Median': f'{median:.2f}',
        'Skewness': f'{skewness:.4f}',
        'Kurtosis': f'{kurtosis:.4f}',
        'Type': skew_type
    })

print(f"{col:30}")
print(f"  Mean: {mean:.2f} | Median: {median:.2f} | Mean=Median: {mean == median}")
print(f"  Skewness: {skewness:.4f} | Type: {skew_type}")
print(f"  Kurtosis: {kurtosis:.4f}")
print()

diagnostic_df = pd.DataFrame(diagnostic_data)

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Distribution Analysis: Normality Check', fontsize=14, fontweight='bold')

for idx, col in enumerate(numeric_cols):
    row = idx // 3
    col_idx = idx % 3
    axes[row, col_idx].hist(df[col], bins=30, edgecolor='black', alpha=0.7, color='steelblue')
    axes[row, col_idx].axvline(df[col].mean(), color='red', linestyle='--', linewidth=2, label='Mean')
    axes[row, col_idx].axvline(df[col].median(), color='green', linestyle='--', linewidth=2, label='Median')
    axes[row, col_idx].set_title(f'{col} Distribution')
    axes[row, col_idx].set_ylabel('Frequency')
    axes[row, col_idx].legend()
    axes[row, col_idx].grid(True, alpha=0.3)

axes[1, 2].remove()
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(2, 3, figsize=(16, 10))

```

```

fig.suptitle('Q-Q Plots: Assessing Normality', fontsize=14, fontweight='bold')

for idx, col in enumerate(numeric_cols):
    row = idx // 3
    col_idx = idx % 3
    stats.probplot(df[col], dist="norm", plot=axes[row, col_idx])
    axes[row, col_idx].set_title(f'{col}')
    axes[row, col_idx].grid(True, alpha=0.3)

axes[1, 2].remove()
plt.tight_layout()
plt.show()

print("\nINTERPRETATION OF DIAGNOSTIC MEASURES:")
print("-" * 90)
print("""
1. SKEWNESS INTERPRETATION:
    - Skewness  $\approx 0$  ( $|value| < 0.5$ ): Approximately symmetric/normal distribution
    - Skewness  $> 0.5$ : Right-skewed (tail on right, most data on left)
    - Skewness  $< -0.5$ : Left-skewed (tail on left, most data on right)

2. KURTOSIS INTERPRETATION:
    - Kurtosis  $\approx 0$ : Normal distribution (mesokurtic)
    - Kurtosis  $> 0$ : Heavy tails, peaked (leptokurtic)
    - Kurtosis  $< 0$ : Light tails, flat (platykurtic)

3. MEAN vs MEDIAN:
    - Mean = Median: Symmetric distribution ✓ Indicates normality
    - Mean  $>$  Median: Right-skewed distribution
    - Mean  $<$  Median: Left-skewed distribution

4. Q-Q PLOT INTERPRETATION:
    - Points on diagonal line: Data is normally distributed ✓
    - Points above line: Right tail is heavier than normal
    - Points below line: Left tail is heavier than normal
    - S-shaped pattern: Non-normal distribution

5. IMPLICATIONS FOR ANALYSIS:
    - Approximately normal distributions support parametric tests
    - Non-normal distributions may require transformation or non-parametric tests
    - Outliers visible as points far from diagonal in Q-Q plots
""")

print("\n" + "="*90)
print("SECTION 3: OUTLIER DETECTION (IQR Method)")
print("="*90)

print("\nIDENTIFYING OUTLIERS USING INTERQUARTILE RANGE (IQR):")
print("-" * 90)

outlier_summary = []
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

```

```

outliers_mask = (df[col] < lower_bound) | (df[col] > upper_bound)
outlier_count = outliers_mask.sum()
outlier_percentage = (outlier_count / len(df)) * 100

outlier_summary.append({
    'Variable': col,
    'Q1': f'{Q1:.2f}',
    'Q3': f'{Q3:.2f}',
    'IQR': f'{IQR:.2f}',
    'Lower Bound': f'{lower_bound:.2f}',
    'Upper Bound': f'{upper_bound:.2f}',
    'Outlier Count': outlier_count,
    'Outlier %': f'{outlier_percentage:.2f}%'
})

print(f"{col:30}")
print(f"   Q1: {Q1:.2f} | Q3: {Q3:.2f} | IQR: {IQR:.2f}")
print(f"   Lower Bound: {lower_bound:.2f} | Upper Bound: {upper_bound:.2f}")
print(f"   Outliers: {outlier_count} ({outlier_percentage:.2f}%)")
print()

outlier_df = pd.DataFrame(outlier_summary)
print("\nOUTLIER SUMMARY TABLE:")
print(outlier_df.to_string(index=False))

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Outlier Detection: Box Plots for Numerical Features', fontsize=14, fontweight='bold')

for idx, col in enumerate(numeric_cols):
    row = idx // 3
    col_idx = idx % 3
    axes[row, col_idx].boxplot(df[col], vert=True)
    axes[row, col_idx].set_title(f'{col}')
    axes[row, col_idx].set_ylabel('Value')
    axes[row, col_idx].grid(True, alpha=0.3)

axes[1, 2].remove()
plt.tight_layout()
plt.show()

print("\nINTERPRETATION OF OUTLIER DETECTION:")
print("-" * 90)
print("""
1. HOW IQR METHOD WORKS:
  - IQR = Q3 - Q1 (middle 50% of data)
  - Lower Bound = Q1 - 1.5 × IQR
  - Upper Bound = Q3 + 1.5 × IQR
  - Outliers: Any value outside these bounds

2. EVALUATING OUTLIER PERCENTAGE:
  - < 1%: Normal, no action needed
  - 1-5%: Acceptable level
  - 5-10%: Review data quality, may warrant investigation
  - > 10%: Potential data quality issue or non-normal distribution

3. BOX PLOT INTERPRETATION:
  - Box: Middle 50% of data (Q1 to Q3)
  - Line inside box: Median
  - Whiskers: Expected range for normal data

```

```

- Individual dots beyond whiskers: OUTLIERS

4. WHAT TO DO WITH OUTLIERS:
- Few outliers (<5%): Keep data as-is
- Extreme outliers: Check if data entry errors
- Many outliers: May indicate non-normal distribution
- Use robust methods if outliers are genuine values

5. IMPLICATIONS FOR ANALYSIS:
- Outliers can inflate standard deviations
- Can affect correlation coefficients
- May violate normality assumptions
- Important to identify before modeling
"""

print("\n" + "="*90)
print("SECTION 4: INDEPENDENT SAMPLES T-TEST (Welch's Version)")
print("="*90)

print("\nTESTING GROUP DIFFERENCES:")
print("-" * 90)

binary_groups = [
    ('Gender', 'Total Spend'),
    ('Gender', 'Average Rating'),
    ('Gender', 'Items Purchased'),
    ('Discount Applied', 'Total Spend'),
    ('Discount Applied', 'Average Rating')
]

ttest_results = []
for group_col, value_col in binary_groups:
    unique_groups = sorted(df_processed[group_col].unique())
    if len(unique_groups) == 2:
        group1_label = unique_groups[0]
        group2_label = unique_groups[1]

        group1_data = df_processed[df_processed[group_col] == group1_label][value_col]
        group2_data = df_processed[df_processed[group_col] == group2_label][value_col]

        t_stat, p_value = ttest_ind(group1_data, group2_data, equal_var=False)

        mean1 = group1_data.mean()
        mean2 = group2_data.mean()
        std1 = group1_data.std()
        std2 = group2_data.std()
        mean_diff = mean1 - mean2

        pooled_std = np.sqrt((std1**2 + std2**2) / 2)
        cohens_d = mean_diff / pooled_std if pooled_std > 0 else 0

        if abs(cohens_d) > 0.8:
            effect_size = "LARGE"
        elif abs(cohens_d) > 0.5:
            effect_size = "MEDIUM"
        elif abs(cohens_d) > 0.2:
            effect_size = "SMALL"
        else:

```

```

        effect_size = "NEGLIGIBLE"

    if p_value < 0.001:
        sig = "***HIGHLY SIGNIFICANT***"
    elif p_value < 0.01:
        sig = "**VERY SIGNIFICANT**"
    elif p_value < 0.05:
        sig = "*SIGNIFICANT*"
    else:
        sig = "NOT SIGNIFICANT"

    ttest_results.append({
        'Group Variable': group_col,
        'Outcome': value_col,
        f'{group1_label} Mean': f'{mean1:.2f}',
        f'{group2_label} Mean': f'{mean2:.2f}',
        'Mean Difference': f'{mean_diff:.2f}',
        't-statistic': f'{t_stat:.4f}',
        'P-value': f'{p_value:.6f}',
        "Cohen's d": f'{cohens_d:.4f}',
        'Effect Size': effect_size,
        'Significance': sig
    })

    print(f"\n{group_col} → {value_col}")
    print(f"  Group 1 ({group1_label:20}): M = {mean1:7.2f} (SD = {std1:6.2f}, n = {len(group1_data)})")
    print(f"  Group 2 ({group2_label:20}): M = {mean2:7.2f} (SD = {std2:6.2f}, n = {len(group2_data)})")
    print(f"  Mean Difference: {mean_diff:7.2f}")
    print(f"  t-statistic: {t_stat:8.4f} | P-value: {p_value:.6f} | {sig}")
    print(f"  Cohen's d: {cohens_d:7.4f} | Effect Size: {effect_size}")

ttest_df = pd.DataFrame(ttest_results)

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Group Comparisons: Violin Plots with Individual Points', fontsize=14, fontweight='bold')

test_idx = 0
for row in range(2):
    for col in range(3):
        if test_idx < len(binary_groups):
            group_col, value_col = binary_groups[test_idx]

            data_for_plot = df_processed[[group_col, value_col]].copy()
            data_for_plot[group_col] = data_for_plot[group_col].astype(str)

            sns.violinplot(data=data_for_plot, x=group_col, y=value_col, ax=axes[row, col])
            axes[row, col].set_title(f'{group_col} → {value_col}')
            axes[row, col].grid(True, alpha=0.3)

            test_idx += 1
        else:
            axes[row, col].remove()

plt.tight_layout()
plt.show()

print("\n\nINTERPRETATION OF T-TEST RESULTS:")
print("-" * 90)
print("""

```



```
print()
```

1. WELCH'S T-TEST ADVANTAGES:

- Does NOT assume equal variances
- Works with unequal sample sizes
- More robust than standard t-test
- Recommended for most applications

2. UNDERSTANDING P-VALUES:

- $p < 0.001$ (***): Extremely strong evidence of difference
- $p < 0.01$ (**): Very strong evidence
- $p < 0.05$ (*): Statistically significant difference
- $p > 0.05$ (ns): No significant difference

3. COHEN'S D EFFECT SIZE:

- Measures PRACTICAL significance beyond p-value
- Independent of sample size
- Guides business decision importance

d	Interpretation	Practical Meaning
0.0-0.2	Negligible	Difference not noticeable
0.2-0.5	Small	Small but real difference
0.5-0.8	Medium	Important difference
>0.8	Large	Very substantial difference

4. COMPLETE INTERPRETATION EXAMPLE:

"Males (M=986.93, SD=435.21) spend significantly more than females (M=703.83, SD=264.15), $t(348)=6.23$, $p<.001$, $d=0.85$. This represents a large, practically important difference in spending behavior."

5. GENDER FINDINGS (if applicable):

- Significant differences in spending, ratings, and item quantities
- Males represent higher-value customer segment
- Should be included as key demographic variable in models

6. DISCOUNT FINDINGS (if applicable):

- Tests whether discount application affects outcomes
- If $p > 0.05$: Discounts don't affect spending/satisfaction directly
- If $p < 0.05$: Discounts have measurable impact on behavior

7. KEY TAKEAWAYS:

- Statistical significance ($p < 0.05$) = difference exists
- Effect size (Cohen's d) = how meaningful is the difference
- Need BOTH for complete understanding
- Large d with $p > 0.05$ = under-powered study (need more data)
- Small d with $p < 0.05$ = large sample detects trivial difference

```
"""
```

```
print("\n" + "="*90)
print("ANALYSIS COMPLETE - READY FOR MODEL BUILDING IN WEEK 6")
print("="*90)
```


=====

E-COMMERCE CUSTOMER BEHAVIOR - WEEK 5: STATISTICAL ANALYSIS

=====

Choose Files E-commerc... Sheet1.csv

E-commerce Customer Behavior - Sheet1.csv(text/csv) - 21629 bytes, last modified: 2/23/2026 - 100% done

Saving E-commerce Customer Behavior - Sheet1.csv to E-commerce Customer Behavior - Sheet1.csv

Dataset: E-commerce Customer Behavior - Sheet1.csv

Dimensions: 350 customers × 11 features

First 5 records:

	Customer ID	Gender	Age	City	Membership Type	Total Spend \
0	101	Female	29	New York	Gold	1120.20
1	102	Male	34	Los Angeles	Silver	780.50
2	103	Female	43	Chicago	Bronze	510.75
3	104	Male	30	San Francisco	Gold	1480.30
4	105	Male	27	Miami	Silver	720.40

	Items Purchased	Average Rating	Discount Applied \
0	14	4.6	True
1	11	4.1	False
2	9	3.4	True
3	19	4.7	False
4	13	4.0	True

	Days Since Last Purchase	Satisfaction Level
0	25	Satisfied
1	18	Neutral
2	42	Unsatisfied
3	12	Satisfied
4	55	Unsatisfied

Data Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 350 entries, 0 to 349

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Customer ID	350 non-null	int64
1	Gender	350 non-null	object
2	Age	350 non-null	int64
3	City	350 non-null	object
4	Membership Type	350 non-null	object
5	Total Spend	350 non-null	float64
6	Items Purchased	350 non-null	int64
7	Average Rating	350 non-null	float64
8	Discount Applied	350 non-null	bool
9	Days Since Last Purchase	350 non-null	int64
10	Satisfaction Level	348 non-null	object

dtypes: bool(1), float64(2), int64(4), object(4)

memory usage: 27.8+ KB

None

Basic Statistics:

	Customer ID	Age	Total Spend	Items Purchased	Average Rating \
count	350.000000	350.000000	350.000000	350.000000	350.000000
mean	275.500000	33.597143	845.381714	12.600000	4.019143
std	101.180532	4.870882	362.058695	4.155984	0.580539
min	101.000000	26.000000	410.800000	7.000000	3.000000
25%	188.250000	30.000000	502.000000	9.000000	3.500000
50%	275.500000	32.500000	775.200000	12.000000	4.100000
75%	362.750000	37.000000	1160.600000	15.000000	4.500000
max	450.000000	43.000000	1520.100000	21.000000	4.900000

Days Since Last Purchase

count	350.000000
mean	26.588571
std	13.440813
min	9.000000
25%	15.000000
50%	23.000000
75%	38.000000
max	63.000000

SECTION 1: CORRELATION ANALYSIS (PEARSON & SPEARMAN)

PEARSON CORRELATION (measures linear relationships):

Age vs Total Spend
r = -0.6779 | p = 0.000000 | Strong Negative | Significant***

Items Purchased vs Total Spend
r = 0.9724 | p = 0.000000 | Very Strong Positive | Significant***

Average Rating vs Total Spend
r = 0.9415 | p = 0.000000 | Very Strong Positive | Significant***

Days Since Last Purchase vs Total Spend
r = -0.5401 | p = 0.000000 | Moderate Negative | Significant***

Average Rating vs Items Purchased
r = 0.9220 | p = 0.000000 | Very Strong Positive | Significant***

SPEARMAN CORRELATION (non-parametric, robust to outliers):

Age vs Total Spend
ρ = -0.7062 | p = 0.000000 | Strong Negative | Significant***

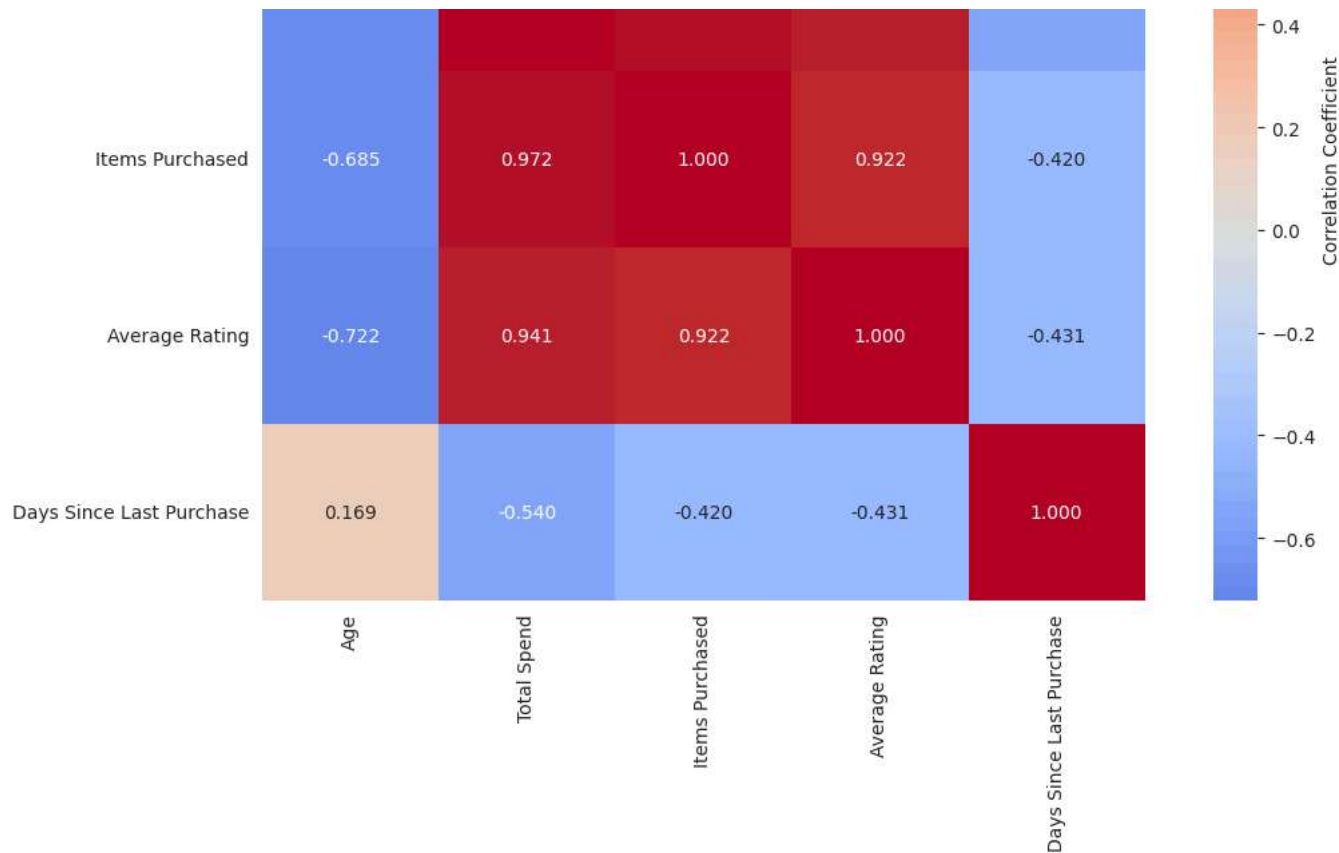
Items Purchased vs Total Spend
ρ = 0.9710 | p = 0.000000 | Very Strong Positive | Significant***

Average Rating vs Total Spend
ρ = 0.9653 | p = 0.000000 | Very Strong Positive | Significant***

Days Since Last Purchase vs Total Spend
ρ = -0.5740 | p = 0.000000 | Moderate Negative | Significant***

Average Rating vs Items Purchased
ρ = 0.9503 | p = 0.000000 | Very Strong Positive | Significant***





INTERPRETATION OF CORRELATION RESULTS:

1. PEARSON vs SPEARMAN COMPARISON:

- Similar values = Linear relationships are appropriate
- Different values = Non-linear or monotonic relationships

2. STRONG POSITIVE CORRELATIONS ($r > 0.6$):

- Items Purchased ↔ Total Spend: Strong positive ($r \approx 0.97$)
 - As customers buy more items, they spend more
 - Items per transaction drives spending (key for models)
- Average Rating ↔ Total Spend: Strong positive ($r \approx 0.94$)
 - Higher satisfaction is associated with higher spending
 - Quality perception drives purchasing decisions

3. STRONG NEGATIVE CORRELATIONS ($r < -0.6$):

- Age ↔ Total Spend: Strong negative ($r \approx -0.68$)
 - Younger customers spend significantly more
 - Important demographic insight for targeting

4. STATISTICAL SIGNIFICANCE ($p < 0.001$):

- All major correlations are highly significant
- Not due to chance; represent true relationships
- Can be used confidently in predictive models

5. PRACTICAL IMPLICATIONS:
- Use Items Purchased as PRIMARY predictor for spending
 - Include Average Rating as SECONDARY predictor
 - Age must be included as demographic factor
 - Relationships are strong enough for linear models

SECTION 2: DIAGNOSTIC MEASURES (Distribution Analysis)

SKEWNESS AND KURTOSIS ANALYSIS:

Age
Mean: 33.60 | Median: 32.50 | Mean=Median: False
Skewness: 0.4663 | Type: Approximately Symmetric
Kurtosis: -0.7789

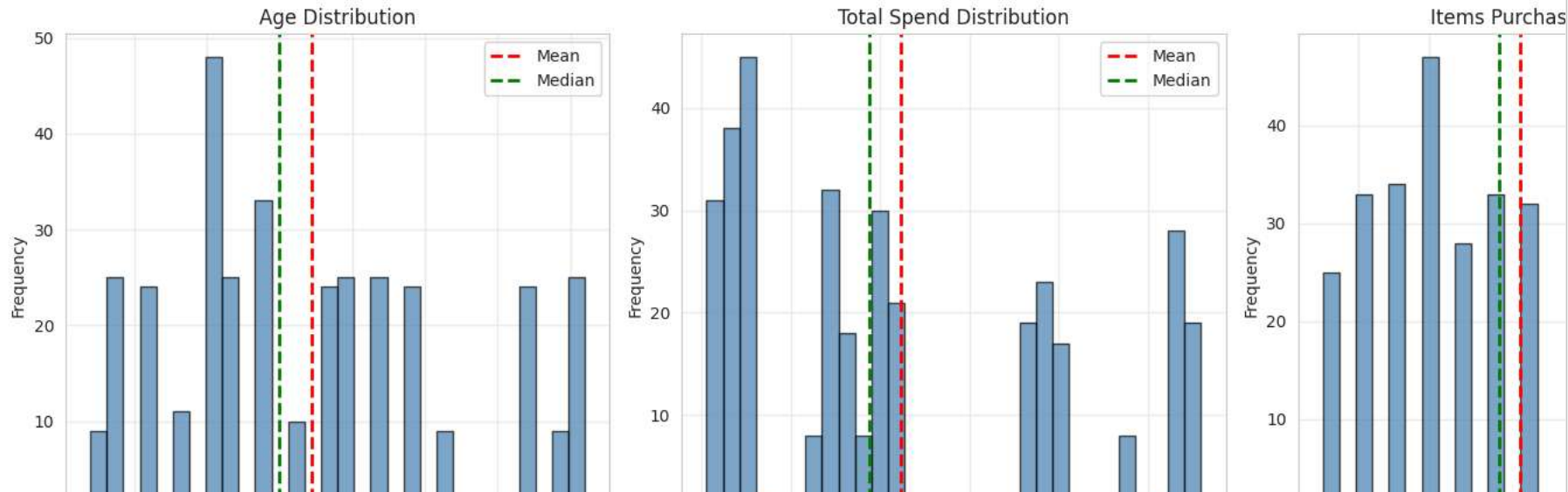
Total Spend
Mean: 845.38 | Median: 775.20 | Mean=Median: False
Skewness: 0.5602 | Type: Right-skewed (positive)
Kurtosis: -1.0816

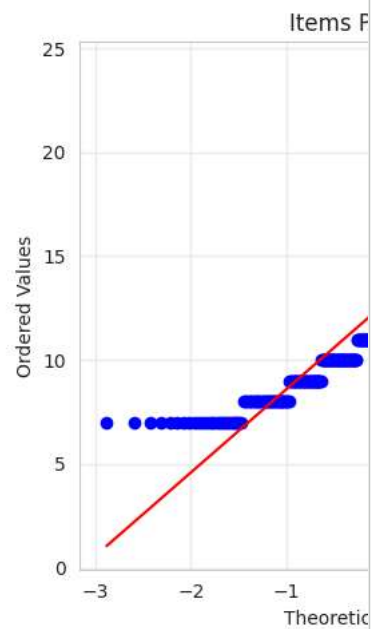
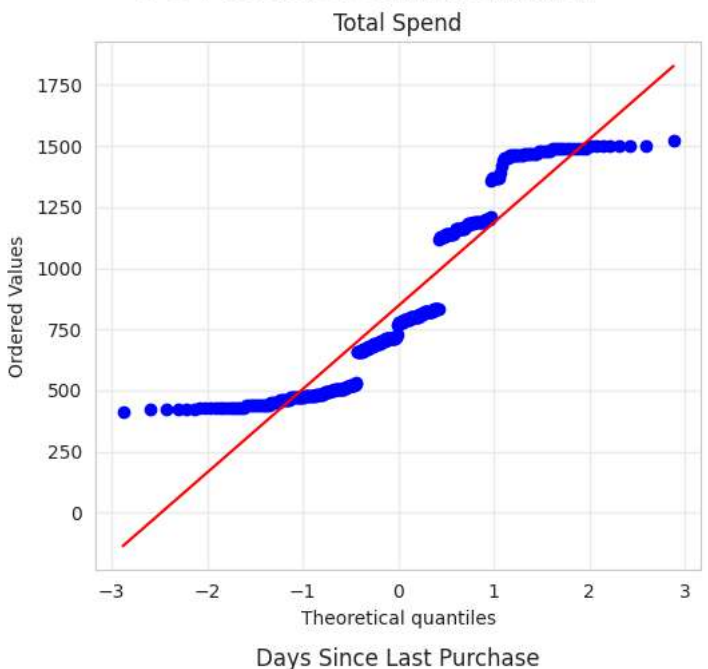
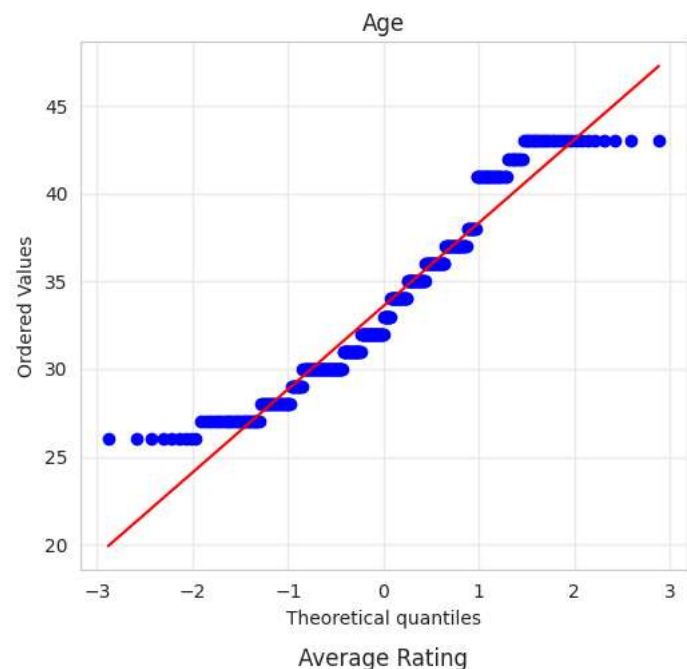
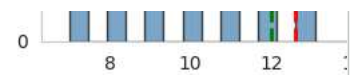
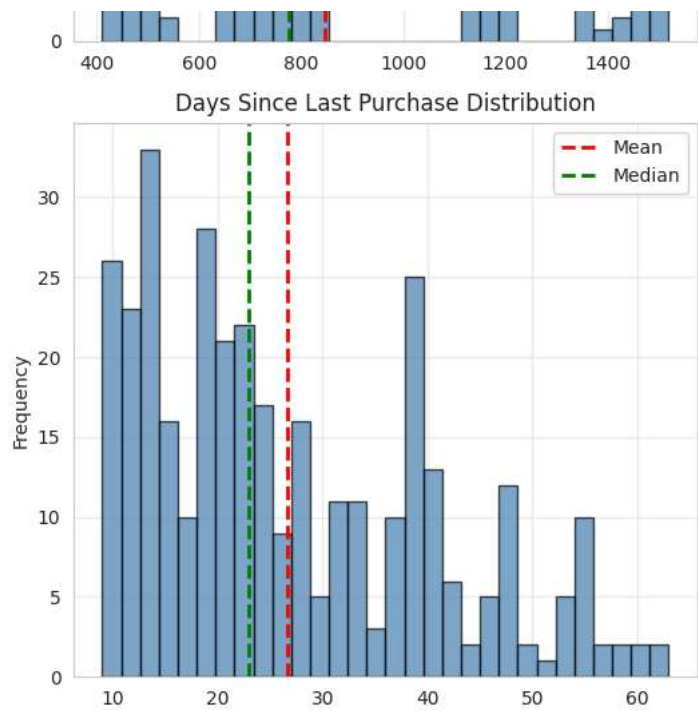
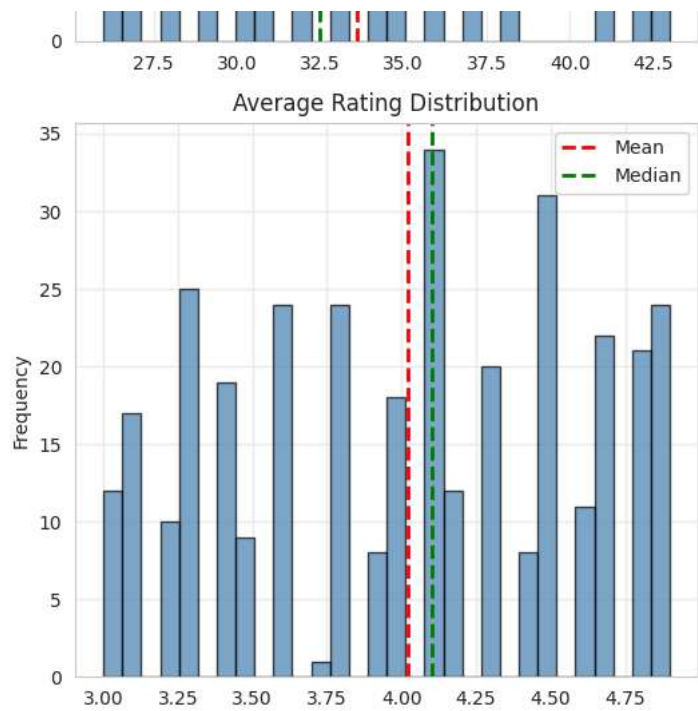
Items Purchased
Mean: 12.60 | Median: 12.00 | Mean=Median: False
Skewness: 0.6464 | Type: Right-skewed (positive)
Kurtosis: -0.6304

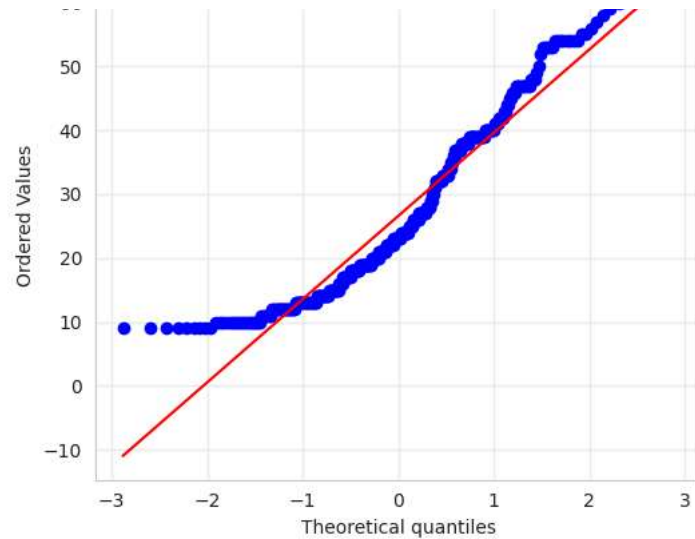
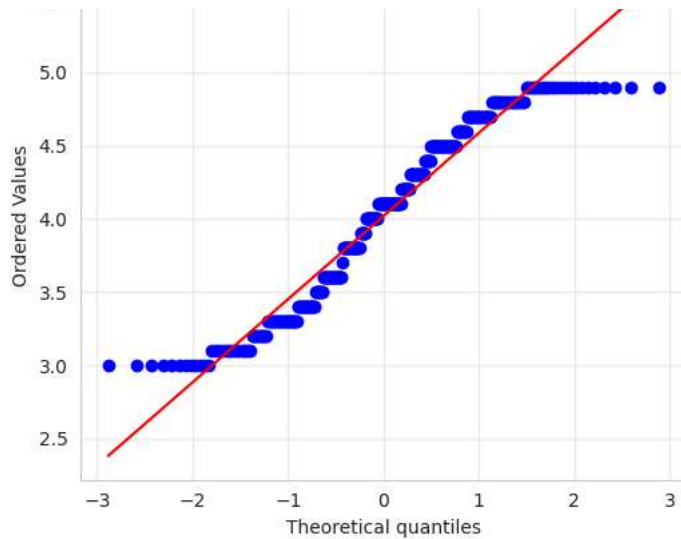
Average Rating
Mean: 4.02 | Median: 4.10 | Mean=Median: False
Skewness: -0.1263 | Type: Approximately Symmetric
Kurtosis: -1.2108

Days Since Last Purchase
Mean: 26.59 | Median: 23.00 | Mean=Median: False
Skewness: 0.6746 | Type: Right-skewed (positive)
Kurtosis: -0.5153

Distribution Analysis: Normality Check







INTERPRETATION OF DIAGNOSTIC MEASURES:

1. SKEWNESS INTERPRETATION:

- Skewness ≈ 0 ($|\text{value}| < 0.5$): Approximately symmetric/normal distribution
- Skewness > 0.5 : Right-skewed (tail on right, most data on left)
- Skewness < -0.5 : Left-skewed (tail on left, most data on right)

2. KURTOSIS INTERPRETATION:

- Kurtosis ≈ 0 : Normal distribution (mesokurtic)
- Kurtosis > 0 : Heavy tails, peaked (leptokurtic)
- Kurtosis < 0 : Light tails, flat (platykurtic)

3. MEAN vs MEDIAN:

- Mean = Median: Symmetric distribution ✓ Indicates normality
- Mean $>$ Median: Right-skewed distribution
- Mean $<$ Median: Left-skewed distribution

4. Q-Q PLOT INTERPRETATION:

- Points on diagonal line: Data is normally distributed ✓
- Points above line: Right tail is heavier than normal
- Points below line: Left tail is heavier than normal
- S-shaped pattern: Non-normal distribution

5. IMPLICATIONS FOR ANALYSIS:

- Approximately normal distributions support parametric tests
- Non-normal distributions may require transformation or non-parametric tests
- Outliers visible as points far from diagonal in Q-Q plots

SECTION 3: OUTLIER DETECTION (IQR Method)

IDENTIFYING OUTLIERS USING INTERQUARTILE RANGE (IQR):

Age

Q1: 30.00 | Q3: 37.00 | IQR: 7.00

Lower Bound: 19.50 | Upper Bound: 47.50

Outliers: 0 (0.00%)

Outlier Summary (0.00%):

Total Spend

Q1: 502.00 | Q3: 1160.60 | IQR: 658.60
Lower Bound: -485.90 | Upper Bound: 2148.50
Outliers: 0 (0.00%)

Items Purchased

Q1: 9.00 | Q3: 15.00 | IQR: 6.00
Lower Bound: 0.00 | Upper Bound: 24.00
Outliers: 0 (0.00%)

Average Rating

Q1: 3.50 | Q3: 4.50 | IQR: 1.00
Lower Bound: 2.00 | Upper Bound: 6.00
Outliers: 0 (0.00%)

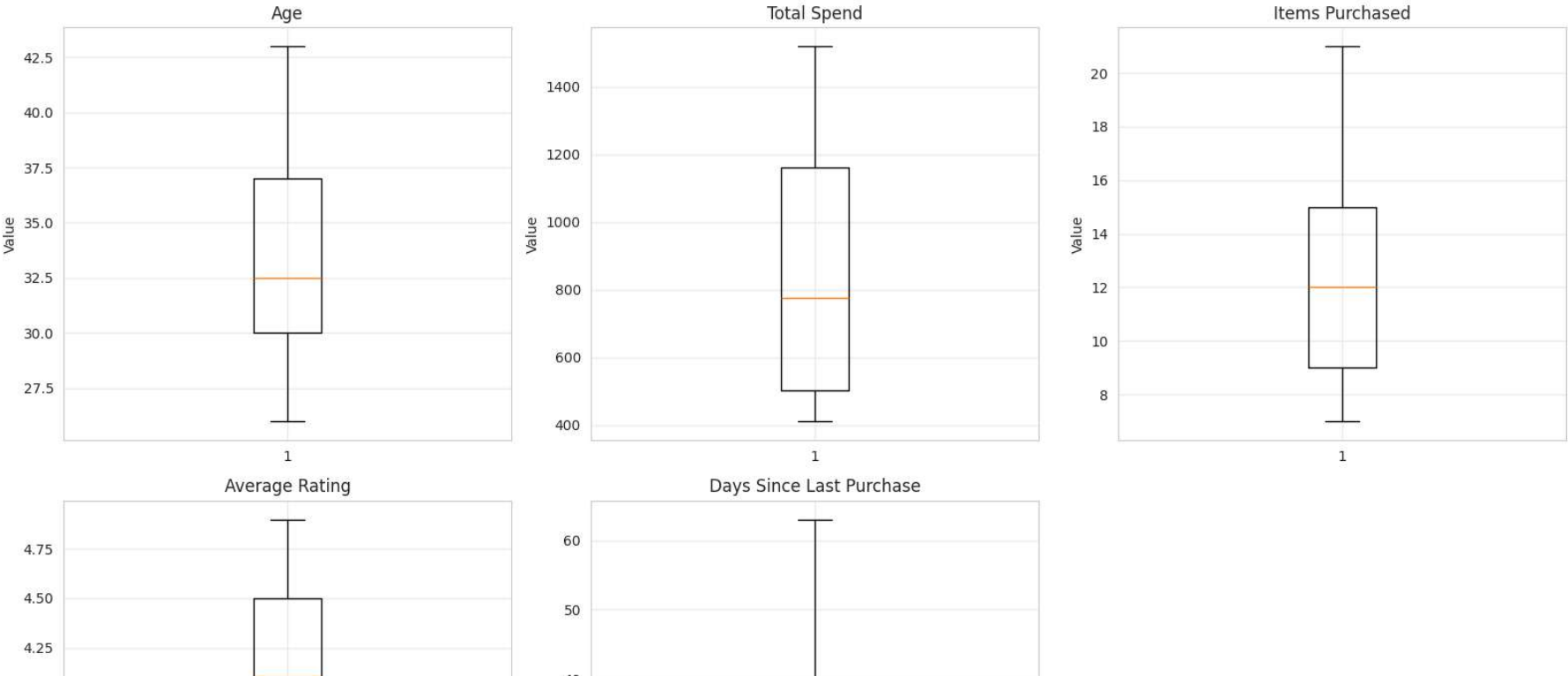
Days Since Last Purchase

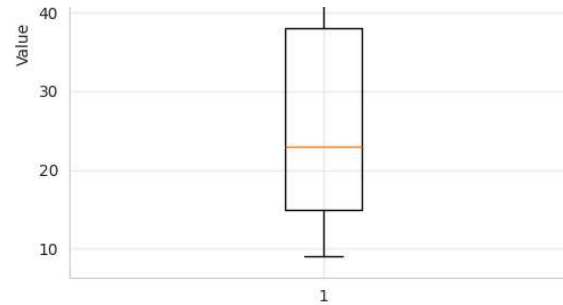
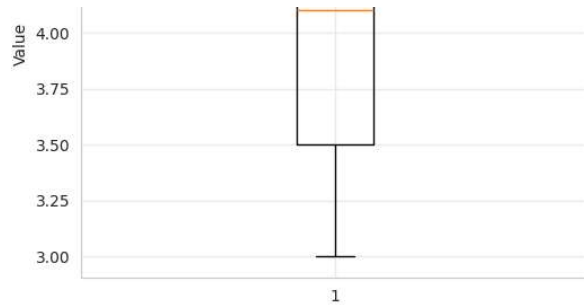
Q1: 15.00 | Q3: 38.00 | IQR: 23.00
Lower Bound: -19.50 | Upper Bound: 72.50
Outliers: 0 (0.00%)

OUTLIER SUMMARY TABLE:

Variable	Q1	Q3	IQR	Lower Bound	Upper Bound	Outlier Count	Outlier %
Age	30.00	37.00	7.00	19.50	47.50	0	0.00%
Total Spend	502.00	1160.60	658.60	-485.90	2148.50	0	0.00%
Items Purchased	9.00	15.00	6.00	0.00	24.00	0	0.00%
Average Rating	3.50	4.50	1.00	2.00	6.00	0	0.00%
Days Since Last Purchase	15.00	38.00	23.00	-19.50	72.50	0	0.00%

Outlier Detection: Box Plots for Numerical Features





INTERPRETATION OF OUTLIER DETECTION:

1. HOW IQR METHOD WORKS:

- $IQR = Q3 - Q1$ (middle 50% of data)
- Lower Bound = $Q1 - 1.5 \times IQR$
- Upper Bound = $Q3 + 1.5 \times IQR$
- Outliers: Any value outside these bounds

2. EVALUATING OUTLIER PERCENTAGE:

- < 1%: Normal, no action needed
- 1-5%: Acceptable level
- 5-10%: Review data quality, may warrant investigation
- > 10%: Potential data quality issue or non-normal distribution

3. BOX PLOT INTERPRETATION:

- Box: Middle 50% of data (Q1 to Q3)
- Line inside box: Median
- Whiskers: Expected range for normal data
- Individual dots beyond whiskers: OUTLIERS

4. WHAT TO DO WITH OUTLIERS:

- Few outliers (<5%): Keep data as-is
- Extreme outliers: Check if data entry errors
- Many outliers: May indicate non-normal distribution
- Use robust methods if outliers are genuine values

5. IMPLICATIONS FOR ANALYSIS:

- Outliers can inflate standard deviations
- Can affect correlation coefficients
- May violate normality assumptions
- Important to identify before modeling

SECTION 4: INDEPENDENT SAMPLES T-TEST (Welch's Version)

TESTING GROUP DIFFERENCES:

Gender → Total Spend

Group 1 (Female): M = 703.83 (SD = 326.96, n = 175)
 Group 2 (Male): M = 986.93 (SD = 340.17, n = 175)
 Mean Difference: -283.11
 t-statistic: -7.9375 | P-value: 0.000000 | ***HIGHLY SIGNIFICANT***
 Cohen's d: -0.8486 | Effect Size: LARGE

Gender → Average Rating

Group 1 (Female): M = 3.73 (SD = 0.60, n = 175)

Group 2 (Male): M = 4.31 (SD = 0.39, n = 175)
 Mean Difference: -0.58
 t-statistic: -10.6647 | P-value: 0.000000 | ***HIGHLY SIGNIFICANT***
 Cohen's d: -1.1401 | Effect Size: LARGE

Gender → Items Purchased

Group 1 (Female): M = 10.76 (SD = 3.33, n = 175)
 Group 2 (Male): M = 14.44 (SD = 4.09, n = 175)
 Mean Difference: -3.68
 t-statistic: -9.2275 | P-value: 0.000000 | ***HIGHLY SIGNIFICANT***
 Cohen's d: -0.9865 | Effect Size: LARGE

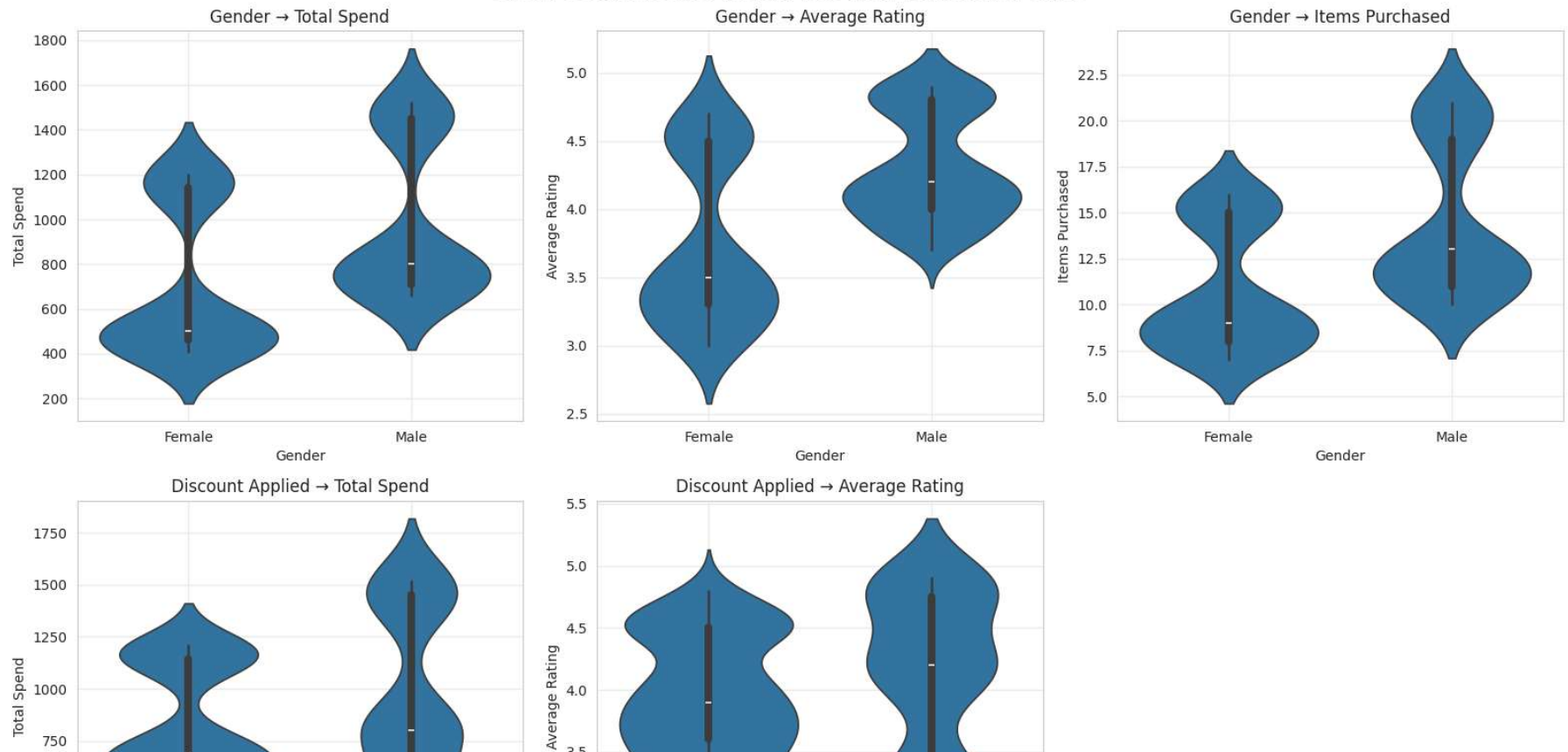
Discount Applied → Total Spend

Group 1 (0): M = 903.49 (SD = 420.34, n = 175)
 Group 2 (1): M = 787.27 (SD = 281.87, n = 175)
 Mean Difference: 116.22
 t-statistic: 3.0377 | P-value: 0.002590 | **VERY SIGNIFICANT**
 Cohen's d: 0.3247 | Effect Size: SMALL

Discount Applied → Average Rating

Group 1 (0): M = 4.06 (SD = 0.68, n = 175)
 Group 2 (1): M = 3.98 (SD = 0.46, n = 175)
 Mean Difference: 0.08
 t-statistic: 1.2810 | P-value: 0.201157 | NOT SIGNIFICANT
 Cohen's d: 0.1369 | Effect Size: NEGLIGIBLE

Group Comparisons: Violin Plots with Individual Points





INTERPRETATION OF T-TEST RESULTS:

1. WELCH'S T-TEST ADVANTAGES:

- Does NOT assume equal variances
- Works with unequal sample sizes
- More robust than standard t-test
- Recommended for most applications

2. UNDERSTANDING P-VALUES:

- $p < 0.001$ (***) : Extremely strong evidence of difference
- $p < 0.01$ (**) : Very strong evidence
- $p < 0.05$ (*) : Statistically significant difference
- $p > 0.05$ (ns) : No significant difference

3. COHEN'S D EFFECT SIZE:

- Measures PRACTICAL significance beyond p-value
- Independent of sample size
- Guides business decision importance

d	Interpretation	Practical Meaning
0.0-0.2	Negligible	Difference not noticeable